

Curs Algoritmi Fundamentali (AF)

Analiza eficienței algoritmilor.

Complexități¹

Conf. dr. Alexandra Băicoianu

Universitatea *Transilvania* din Brașov
Facultatea de Matematică și Informatică

2025

¹Mulțumesc studenților mei pentru corecturile aduse acestui material.

Agenda

- 1 Recapitulare noțiuni curs anterior
- 2 Analiza asimptotică: O (Omicron), Ω (Omega), Θ (Theta)
 - Omicron
 - Omega
 - Theta
- 3 Exerciții diverse



Recapitulare noțiuni curs anterior

- număr de operații ($T(n)$)
- termen dominant
- ordin de creștere
- etape în analiza eficienței
- cazul cel mai favorabil și cazul cel mai defavorabil
- complexitate constantă, logaritmică, liniară, pătratică, etc

Analiza asimptotică: O (Omicron), Ω (Omega), Θ (Theta)

Analiza timpilor de execuție pentru **dimensiuni mici** ale problemei nu permite diferențierea algoritmilor eficienți de cei ineficienți. Diferențele dintre ordinele de creștere devin relevante pe parcursul creșterii dimensiunii problemei.

Analiza asimptotică are ca obiectiv studiul proprietăților timpului de execuție atunci când **dimensiunea problemei tinde către infinit**.

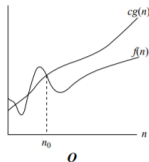
Există trei notații folosite pentru diverse clase de eficiență: O (Omicron), Ω (Omega) și Θ (Theta).

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei. Spunem că funcțiile sunt în relația:

$$f(n) = O(g(n))[*]^2$$

dacă și numai dacă există $n_0 \in \mathbb{N}$, $c > 0$ astfel încât $0 \leq f(n) \leq c \cdot g(n)$, pentru orice $n > n_0$.

Pentru valori mari ale lui n , $f(n)$ este mărginită superior de $g(n)$ înmulțită cu o constantă pozitivă:



²Abuz de notație conform *Introduction to Algorithms*, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

Diferența dintre $f(n) = O(g(n))$ și $f(n) \in O(g(n))$

$f(n) = O(g(n))$: "abuz de notație"

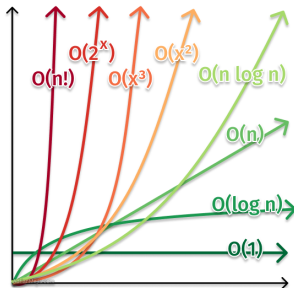
- Deși folosim semnul egal, el nu denotă o egalitate obișnuită, ci mai degrabă o afirmație că "funcția $f(n)$ aparține clasei de complexitate $O(g(n))$ ".
- Este o modalitate concisă de a spune că $f(n)$ este limitată superior de $g(n)$.
- Este o notație folosită pentru a descrie comportamentul limită al unei funcții în raport cu alta pe măsură ce n tinde către infinit.
- Formal, $f(n) = O(g(n))$ dacă există constante pozitive c și n_0 astfel încât $0 \leq f(n) \leq c \cdot g(n)$ pentru toate valorile $n \geq n_0$.

Diferența dintre $f(n) = O(g(n))$ și $f(n) \in O(g(n))$

$f(n) \in O(g(n))$:

- Aceasta este notația matematic corectă și riguroasă.
- Aceasta indică apartenența funcției $f(n)$ la clasa de complexitate $O(g(n))$. Notația $O(g(n))$ definește o mulțime (sau clasă) de funcții. Această mulțime include toate funcțiile care cresc cel mult la fel de repede ca $g(n)$.
- De obicei, se folosește într-un context mai general pentru a indica că funcția $f(n)$ are un anumit tip de complexitate și se încadrează în clasa $O(g(n))$.
- Formal, $f(n) \in O(g(n))$ indică că funcția $f(n)$ aparține mulțimii de funcții pentru care există o funcție $g(n)$ și o constantă c astfel încât $0 \leq f(n) \leq c \cdot g(n)$ pentru toate valorile n suficient de mari.

Exemple complexități



Tim constant	$O(1)$
Tim logaritmic	$O(\log n)$
Tim liniar	$O(n)$
Tim quasiliniar/linearithmic	$O(n \log n)$
Tim patratic	$O(n^2)$
Tim exponential	$O(C^n)$
Tim factorial	$O(n!)$

O clasificare a algoritmilor folosind notația O :

$$\begin{aligned}
 &O(1) \subset O(\log n) \subset O(\log^k n) \\
 &\subset O(n) \subset O(n^2) \subset \dots \subset O(n^{k+1}) \subset O(2^n) \dots
 \end{aligned}$$

Exemple

Exemplu 1: Considerăm un algoritm care rezolvă o problemă de căutare într-un set de date sortat. Timpul său de execuție este $O(\log n)$, unde n reprezintă dimensiunea setului de date. Acesta indică faptul că timpul de execuție crește în mod logaritmic odată cu mărimea setului de date. Algoritmii cu această complexitate, precum **căutarea binară**, sunt eficienți pentru seturi de date mari.

Exemplu 2: Un alt algoritm, care are un timp de execuție $O(n^2)$, poate fi asociat cu un algoritm de sortare ineficient, cum ar fi **Bubble Sort**. În acest caz, creșterea dimensiunii setului de date duce la o creștere pătratică a timpului de execuție.

Notăția O (Omicron) - Observații și alte exemple

O este folosită pentru a descrie o margine superioară, ea este utilizată pentru a mărgini cazul cel mai nefavorabil de execuție al unui algoritm. Notăția pune în evidență o **margine superioară a complexității algoritmului** în aceeași măsură pentru orice intrare.

O variantă alternativă pentru definirea notației O este: $f(n) \in O(g(n))$ dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ este 0 ($f(n)$ crește strict mai lent decât $g(n)$) sau o constantă finită și pozitivă. Limita nu poate să fie ∞ .

Metoda cu limite este o consecință a definiției uzuale, utilă pentru cazurile în care limita există.

O (Omicron) Exerciții deducție matematică

Enunț: Să se demonstreze următoarele relații legate de notația O :

- $\log n = O(n)$
- $2^{n+1} = O\left(\frac{3^n}{n}\right), \forall n \geq 7$

Item 1: Dorim să demonstrăm că: $\forall n > 1 \log n \leq n$. Pentru $n = 1$ este trivial, cum $0 < 1$. Presupunem că $n \geq 1$ și că $\log n \leq n$. De aceea, vom avea: $\log(n+1) \leq \log(2n) = \log n + 1 \leq n + 1$, ceea ce trebuie demonstrat.

Item 2: Luăm $n = 7$, cum $2^8 = 256$ și $\frac{3^7}{7} > 312$, ceea ce trebuie demonstrat este adevărat. Presupunem $n \geq 7$ și $2^{n+1} \leq \frac{3^n}{n}$. Ceea ce trebuie demonstrat este: $2^{n+2} \leq \frac{3^{n+1}}{n+1}$. **Cum rezolvați?**

O (Omicron) Exerciții grilă

Enunț: Folosind definiția notației O , care reprezintă o limită superioară asimptotică, identificați toate clasele de complexitate asimptotică (de tip O) dintre opțiunile de mai jos, cărora le aparține funcția $\frac{3}{2} * n$

a. $O(1)$ b. $O(n)$ c. $O(n^2)$ d. $O(n^3)$ e. $O(2^n)$

Observație: Funcția $\frac{3}{2} * n$ aparține tuturor claselor de complexitate care reprezintă o limită superioară pentru creșterea sa. Dacă o funcție este $O(n)$, atunci ea este implicit și $O(n^2)$, $O(n^3)$, $O(2^n)$ și așa mai departe, deoarece $n < n^2 < n^3 < 2^n$ pentru n suficient de mare.

O (Omicron) Exerciții grilă

Enunț: Care dintre următoarele NU este $O(n^2)$?

A. $10n^2 + 20n + 30$

B. $n^{1.5} + 2n^{1.2} + 5$

C. 2^n

D. $n \log n$

Rezolvare: C este varianta incorectă deoarece este o funcție exponențială și nu este de ordinul $O(n^2)$.

O (Omicron) Exerciții grilă

Enunț: Folosind definiția notației O , care reprezintă o limită superioară asimptotică, identificați toate clasele de complexitate asimptotică (de tip O) dintre opțiunile de mai jos, cărora le aparține funcția: $n^2 * 2^n$.

- A. n
- B. n^2
- C. n^3
- D. 2^n
- E. niciuna dintre variantele de mai sus

$n^2 * 2^n$ crește mai rapid sau nu decât 2^n ?

O (Omicron) Exerciții Adevărat/Fals

Enunț: Adevărat sau fals? Justificați (folosind varianta cu definiție sau alternativă (calcularea limitei))

a. $n^2 \in O(n^3)$

b. $n^3 \in O(n^2)$

c. $2^n \in O(n!)$

d. $(n + m)^2 \in O(n^2 + m^2)$

e. $3^n \in O(2^n)$

f. $\log_2 3^n \in O(\log_2 2^n)$

O (Omicron) Exerciții Adevărat/Fals - Rezolvare

a. Adevărat. Dacă există constante pozitive c și n_0 astfel încât $0 \leq f(n) \leq c * g(n)$ pentru toate $n \leq n_0$. În cazul nostru, $n^2 \leq c * n^3$ este satisfăcut pentru $c = 1$ și $n_0 = 1$.

b. Fals. Dacă $f(n) \in O(g(n))$ atunci $g(n)$ ar trebui să fie o funcție care crește cel puțin la fel de rapid ca $f(n)$. Dar n^3 crește mai rapid decât n^2 .

c. De rezolvat.

d. Adevărat. Putem folosi inegalitatea $(n + m)^2 \leq 2(n^2 + m^2)$ pentru a demonstra acest lucru. Definiție: $f(n, m) \in O(g(n, m))$ dacă există constante pozitive c și n_0 astfel încât $0 \leq f(n, m) \leq c * g(n, m)$ pentru toate $n, m \geq n_0$

e. De rezolvat.

f. Adevărat. $\log_2 3^n \in O(\log_2 2^n)$ este $O(\log_2 2^n)$ deoarece 3^n este mai mare decât 2^n și logaritmul în baza 2 este o funcție strict crescătoare.

O (Omicron) Exerciții

Enunț: Consideră două funcții definite pe mulțimea numerelor reale pozitive:

$$f(x) = 3x^2 + 2x + 1$$

$$g(x) = 2x^2 + 5x$$

Demonstrează că $f(x) = O(g(x))$ folosind definiția formală a notației O .

O (Omicron) Exerciții - Rezolvare

Definiția formală a O afirmă că $f(x) = O(g(x))$ dacă există constante pozitive c și x_0 astfel încât $0 \leq f(x) \leq c \cdot g(x)$ pentru fiecare $x \geq x_0$.

Pasul 1: Alegerea constantei c și x_0

Alegem $c = 4$ și $x_0 = 2$. Avem nevoie să demonstrăm că $0 \leq f(x) \leq 4 \cdot g(x)$ pentru fiecare $x \geq 2$.

Pasul 2: Demonstrația inegalităților

Cazul de bază: $x = 2$

$$f(2) = 3 \cdot 2^2 + 2 \cdot 2 + 1 = 15$$

$$g(2) = 2 \cdot 2^2 + 5 \cdot 2 = 18$$

Verificăm că $0 \leq 15 \leq 4 \cdot 18$, ceea ce este adevărat.

Pasul inductiv: $x \geq 2$

$$f(x) = 3x^2 + 2x + 1$$

$$g(x) = 2x^2 + 5x$$

O (Omicron) Exerciții - Rezolvare (Continuare)

Trebuie să arătăm că $0 \leq 3x^2 + 2x + 1 \leq 4 \cdot (2x^2 + 5x)$.

Inegalitatea din stânga:

$$0 \leq 3x^2 + 2x + 1$$

Adăugăm $-3x^2 - 2x - 1$ la ambele părți ale inegalității, obținem $-3x^2 - 2x \leq 0$, iar aceasta este adevărată, deoarece x este pozitiv sau zero pe domeniul de definiție.

Inegalitatea din dreapta:

$$3x^2 + 2x + 1 \leq 4 \cdot (2x^2 + 5x)$$

$$3x^2 + 2x + 1 \leq 8x^2 + 20x$$

$$0 \leq 5x^2 + 18x - 1$$

Această inegalitate este satisfăcută pentru orice $x \geq 2$

Astfel, avem $0 \leq f(x) \leq 4 \cdot g(x)$ pentru fiecare $x \geq 2$, ceea ce demonstrează că $f(x) = O(g(x))$.

Notăția Ω (Omega)

Desemnează **marginea asimptotică inferioară** a unei funcții.

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei.

Spunem că funcțiile sunt în relația:

$$f(n) = \Omega(g(n))$$

dacă și numai dacă

$$\exists n_0 \in \mathbb{N}, \exists c > 0 \mid f(n) \geq c \cdot g(n) \geq 0, \forall n \geq n_0$$

Funcția $f(n)$ **este marginită inferior de $g(n)$ înmulțită eventual cu o constantă pozitivă.**

Notăția Ω (Omega)

Funcțiile $f(n) = n^4 + 2n^2 + 10n + 500$ și $g(n) = n^2$. Conform definiției, dacă găsim constante pozitive pentru care să se îndeplinească relația, atunci $f(n) = \Omega(g(n))$. Putem alege $c = 50$ și obținem valorile din tabelul:

n	$f(n)$	$50 * g(n)$
1	513	50
3	629	450
4	828	800
5	1225	1250
7	3069	2450
10	10800	5000

Putem concluda că pentru orice $n_0 \geq 7$, vom avea $0 \leq 50 * n^2 \leq n^4 + 2n^2 + 10n + 500$, deci avem $f(n) = \Omega(n^2)$.

Ω - observații

Câteva observații:

- **O definiție alternativă** a notației Ω este: $f(n) \in \Omega(g(n))$ dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ este ∞ ($f(n)$ crește semnificativ mai rapid decât $g(n)$) sau o constantă finită și pozitivă diferită de 0 ($f(n)$ și $g(n)$ cresc la aceeași rată (până la o constantă multiplicatoare).
- Se păstrează proprietățile următoare: **reflexivitate, tranzitivitate și funcții polinomiale.**
- Pentru o funcție $f(n)$ **pot exista mai multe funcții** $g(n)$, astfel încât $f(n) = \Omega(g(n))$.
- Atunci când folosim notația $\Omega(n)$ să încercăm să găsim cea mai mare valoare posibilă pentru $g(n)$, astfel încât să avem o idee mai clară despre **cea mai bună margine inferioară.**

Notăția Θ (Theta)

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei (aceasta trebuie să aibă o dimensiune mare). Se află în relația:

$$f(n) = \Theta(g(n))$$

dacă și numai dacă

$$\exists n_0 \in \mathbb{N}, \exists c_1, c_2 > 0$$

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n), \forall n \geq n_0$$

Notăția se folosește între funcțiile f și g în contextul în care $f(n)$ are același ritm de creștere ca al lui $g(n)$. Altfel spus, oricare două constante pozitive am alege, funcția $f(n)$ este marginită de către cea $g(n)$ și superior, dar și inferior.

Notăția Θ (Theta)

De exemplu funcțiile $f(n) = n^4 + 2n^2 + 10n + 500$ și $g(n) = n^4$. Dacă există $c_1, c_2 > 0$, atunci $f(n) = \Theta(g(n))$. Găsim constantele pozitive c_1, c_2 și n_0 astfel încât funcția să fie delimitată corect și fără a fi necesară studierea cazurilor particulare. Astfel, dacă alegem valorile $c_1 = 0.3$ și $c_2 = 5$ obținem:

n	$0.3 * n^4$	$f(n)$	$5 * n^4$
1	0.3	513	5
2	4.8	544	80
3	24.3	629	405
4	76.8	828	1280
5	187.5	1225	3125

Putem spune că pentru orice $n_0 \geq 4$ avem $f(n) = \Theta(n^4)$. Așadar, $\Theta(g(n))$ înseamnă că **funcția este îngrădită de $g(n)$** .

Notăția Θ (Theta)

Câteva observații:

- Dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ este o constantă finită și pozitivă ($f(n)$ și $g(n)$ cresc la exact aceeași rată asimptotică) diferită de 0 și de ∞ atunci $f(n) \in \Theta(g(n))$
- Pentru o funcție $f(n)$ există **o singură funcție** $g(n)$ pentru care $f(n) = \Theta(g(n))$, fapt datorat constantelor.
- Ori de câte ori este posibil, vom utiliza notația Θ pentru complexitatea unui algoritm
- $\Theta(g(n)) \subset O(g(n))$ și $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$, pe scurt $\Theta(g(n))$ este "cel mai specifică".

Concluzii - Notățiile asimptotice și limitele matematice

Definițiile notațiilor asimptotice sunt asemănătoare cu cele ale unei limite. Ca urmare, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ se poate utiliza pentru stabilirea relației asimptotice corecte dintre cele două funcții.

Astfel, avem:

- Dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0, \infty \Rightarrow f = \Theta(g)$ - ritm de creștere asemănător
- Dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty \Rightarrow f = O(g)$ - $f(n)$ crește cel mult la fel de rapid ca $g(n)$
- Dacă $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0 \Rightarrow f = \Omega(g)$ - $f(n)$ crește cel puțin la fel de rapid ca $g(n)$

Se poate folosi și regula lui L'Hospital, doar dacă $\lim_{n \rightarrow \infty} f(n) = \infty$ și $\lim_{n \rightarrow \infty} g(n) = \infty$ (dacă limita este de o formă nedeterminată, cum ar fi $\frac{0}{0}$ sau $\frac{\infty}{\infty}$).

Exerciții diverse

Enunț: Care dintre următoarele NU este $O(n^2)$?

A. $15^{10} * n + 120999$

B. $n^{1.98}$

C. $n^3 / \sqrt{n}$

D. $2^{20} * n$

Rezolvare: Răspuns corect C. Pentru a **NU** face parte din $O(n^2)$, ordinul de creștere trebuie să fie mai mare decât n^2 .

Exerciții diverse

Enunț: Care dintre următoarele reprezintă $\Theta(n)$?

A. $5 * n + 10$

B. $n * \log n + 2 * n$

C. $n^2 + 2 * n$

D. $100 * n - 500$

Rezolvare: Răspunsuri corecte A. D. De exemplu, punctul C este incorect. Dacă calculăm limita: $\lim_{n \rightarrow \infty} \frac{n^2 + 2n}{n} = \lim_{n \rightarrow \infty} (n + 2)$, pe măsură ce $n \rightarrow \infty$, $n + 2 \rightarrow \infty$, deci limita este ∞ . Deoarece limita este ∞ , $n^2 + 2n \notin \Theta(n)$. (Aceasta ar fi în $\Theta(n^2)$).

Exerciții diverse

```
int fibo(int n) {  
    int i, j, k, t;  
    j = 1; i = 0;  
    for (k = 1; k <= n + 1; k++) {  
        t = i + j;  
        i = j; j = t;  
    }  
    return j;  
}
```

Enunț: Fie bucata de cod de mai sus. Cât este complexitatea de spațiu suplimentară a algoritmului?

a. $\Theta(1)$ b. $\Theta(n)$ c. $\Theta(n^2)$ d. $\Theta(n^3)$

Rezolvare: Răspuns corect A. În acest caz, mai precis și direct este să utilizăm notația Θ (memoria este întotdeauna constantă, nu doar că este limitată superior de o constantă), și nu O .

Exerciții diverse

Enunț: Care este complexitatea în notația Θ pentru următoarea bucată de cod:

```

Subalgoritm function(n):
    i ← n/2
    cât timp i ≤ n execută
        j ← 1
        cât timp j + n/2 ≤ n execută
            k ← 1
            cât timp k ≤ n execută
                k ← k * 2
            sf_cât timp
            j ← j + 1
        sf_cât timp
        i ← i + 1
    sf_cât timp
sf_subalgoritm
  
```

Exerciții diverse

Enunț: Care este complexitatea în notația Θ pentru următoarea bucată de cod:

```
Subalgoritm function(n):
    i ← n/2
    cât timp i ≤ n execută
        j ← 1
        cât timp j ≤ n execută
            k ← 1
            cât timp k ≤ n execută
                k ← k * 2
            sf_cât timp
            j ← j * 2
        sf_cât timp
        i ← i + 1
    sf_cât timp
sf_subalgoritm
```


Exerciții diverse

Enunț: Stabiliți funcția potrivită pentru: $f(n) = 100n + \log n$ și $g(n) = n + (\log n)^2$.

Rezolvare: Calculăm limita:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

$$= \lim_{n \rightarrow \infty} \frac{100n + \log n}{n + (\log n)^2} = \lim_{n \rightarrow \infty} \frac{100 + \frac{\log n}{n}}{1 + \frac{(\log n)^2}{n}} = 100.$$

Cum rezultatul este o constantă și ritmul este același, deducem că relația corectă este: $f(n) \in \Theta(g(n))$.

Enunț: Stabiliți funcția potrivită și pentru funcțiile $f(n) = \log n$ și $g(n) = \log n^2$.

Exerciții diverse

Enunț: Stabiliți funcția potrivită pentru funcțiile f și g din tabel. Justificați fiecare funcție aleasă.

$f(n)$	$g(n)$	$f(n) = \dots g(n)$
$n^3 + 3n + 1$	n^4	
$\log n$	n	
$n \cdot 2^n$	3^n	
n	$\log^5 n$	
$\log n$	$\log n^2$	
$100n + \log n$	$n + \log^2 n$	